

APÊNDICE A – Documentação Técnica

Este documento descreve a arquitetura técnica e os componentes do Sistema de Controle de Coleta de Resíduos Sólidos Urbanos. O projeto foi desenvolvido como parte do Trabalho de Conclusão de Curso (TCC) para o curso de Sistemas de Informação da Universidade Federal de Viçosa - Campus Rio Paranaíba (UFV-CRP).

O código-fonte do projeto é aberto e está disponível para consulta no *GitHub* <<https://github.com/luissaster/sistema-coleta-rsu>>, estando sob a Licença MIT.

A.1 Requisitos de Ambiente

Para executar o projeto em um ambiente de desenvolvimento local, os únicos pré-requisitos são:

- **Docker**: O motor de *containerização* para executar a aplicação.
- **Docker Compose**: A ferramenta para definir e executar aplicações *Docker*. Geralmente já vem instalado com o *Docker Desktop* para *Windows* e *Mac* (<<https://www.docker.com/products/docker-desktop/>>).

Todas as dependências, como *Python*, *Django*, *Node.js*, *React* e o SGBD *PostgreSQL*, são gerenciadas e executadas dentro de *containers Docker*, conforme definido nos arquivos *Dockerfile* e *docker-compose.yml*. Isso garante um ambiente de desenvolvimento consistente e elimina a necessidade de instalar qualquer uma dessas tecnologias diretamente na máquina *host*.

A.2 Arquitetura Geral do Sistema

O sistema foi projetado com uma arquitetura baseada em serviços, *containerizada* com *Docker* e orquestrada pelo *Docker Compose*. Esta abordagem desacopla os componentes principais da aplicação, promovendo modularidade, escalabilidade e facilidade de manutenção. A comunicação entre os serviços é realizada através de uma rede *Docker* privada, garantindo isolamento e segurança.

Os principais serviços que compõem a arquitetura são:

- **Back-end (Django)**: O cérebro da aplicação, construído com o *framework Django*. É responsável por toda a lógica de negócio, processamento de dados, autenticação

de usuários e pela exposição de uma API REST para consumo pelo *front-end* e outros clientes.

- **Front-end (React):** Uma *Single-Page Application* (SPA) desenvolvida com *React*, que oferece uma interface de usuário rica, interativa e responsiva. Consome os dados da API do *back-end* para apresentar as informações e permitir a interação do usuário com o sistema.
- **Banco de Dados (PostgreSQL com PostGIS):** Um sistema de gerenciamento de banco de dados relacional *PostgreSQL*, estendido com o *PostGIS*. Essa extensão é crucial, pois adiciona suporte para tipos de dados geográficos e funções espaciais, permitindo o armazenamento e a consulta eficiente de geometrias como rotas (*LineString*) e coordenadas de pontos de coleta (*Point*).
- **Cache e Fila de Tarefas (Redis) e Workers (Celery):** Os serviços *Redis* (*broker* de mensagens) e *Celery* (*workers* para tarefas assíncronas e agendadas) estão incluídos na arquitetura e no `docker-compose.yml`. Embora não haja funcionalidades assíncronas implementadas na versão atual, a estrutura está pronta para facilitar a adição de trabalhos em segundo plano no futuro, como a geração de relatórios ou o envio de notificações, sem a necessidade de reconfigurar a arquitetura base.

A.2.1 Arquitetura em Três Camadas

A estrutura do sistema segue o padrão de arquitetura em três camadas (*3-Tier Architecture*), que divide a aplicação em:

- **Camada de Apresentação (Presentation Layer):** Composta pelo *front-end* em *React*. É a interface com o usuário, responsável por exibir os dados e capturar as interações. Ela não contém lógica de negócio, apenas a lógica de apresentação e a comunicação com a camada de aplicação via chamadas de API.
- **Camada de Aplicação (Application/Logic Layer):** Implementada pelo *back-end* em *Django*. Esta camada centraliza toda a lógica de negócio, regras de validação, processamento de dados e coordenação das operações. Ela expõe os serviços através de uma API REST, servindo como ponte entre a apresentação e os dados.
- **Camada de Dados (Data Layer):** Consiste no banco de dados *PostgreSQL* com a extensão *PostGIS*. É responsável pela persistência e gerenciamento dos dados da aplicação. O acesso a esta camada é controlado exclusivamente pela camada de aplicação, garantindo a integridade e a segurança dos dados.

Esta separação de responsabilidades torna o sistema mais organizado, flexível e fácil de escalar. Cada camada pode ser desenvolvida, testada e atualizada de forma independente das outras.



Figura 27 – Diagrama da arquitetura do sistema.

A.2.2 Containerização e Orquestração

Cada serviço é encapsulado em um *container Docker*, com seu ambiente e dependências definidos em um *Dockerfile* específico. O arquivo `docker-compose.yml` na raiz do projeto é o responsável por definir e orquestrar a interação entre esses containers. Ele configura:

- **Redes:** Uma rede customizada (`residuos_network`) é criada para permitir a comunicação entre os *containers* pelos seus nomes de serviço.
- **Volumes:** Volumes são utilizados para persistir dados importantes, como os dados do banco de dados (`postgres_data`) e os arquivos de mídia enviados pelos usuários (`media`).
- **Variáveis de Ambiente:** Configurações sensíveis ou que variam entre ambientes (desenvolvimento, produção) são injetadas nos *containers* através de variáveis de ambiente, como credenciais de banco de dados e chaves secretas.

A.2.2.1 Comandos *Docker Compose*

A interação com o ambiente de desenvolvimento *containerizado* é feita principalmente através de comandos `docker compose`. Abaixo estão os comandos mais comuns para o gerenciamento do ciclo de vida da aplicação:

- **Construir e iniciar os serviços:**

```
docker compose up --build
```

Este comando constrói as imagens dos *containers* (se ainda não existirem ou se os *Dockerfiles* foram alterados) e inicia todos os serviços definidos no `docker-compose.yml`. A *flag* `-d` pode ser adicionada para executar em modo “*detached*” (em segundo plano).

- **Parar e remover os serviços:**

```
docker compose down
```

Para e remove os *containers*, redes e volumes criados pelo `up`. Adicionar a *flag* `-v` remove também os volumes nomeados.

- **Listar *containers* em execução:**

```
docker compose ps
```

Mostra o *status* de todos os *containers* associados ao projeto.

- **Visualizar *logs* de um serviço:**

```
docker compose logs -f backend
```

Exibe os *logs* em tempo real do serviço especificado (neste caso, o `backend`). É extremamente útil para depuração.

- **Executar um comando em um *container*:**

```
docker compose exec backend python manage.py shell
```

Abre um *shell* interativo dentro de um *container* em execução. Neste exemplo, inicia um *shell* do *Django* dentro do *container* `backend`, permitindo a execução de comandos de gerenciamento.

A.3 *Back-end*

O *back-end* é uma aplicação modularizada, construída em *Python* com o *framework* *Django* e o *Django REST Framework* (DRF).

A.3.1 Tecnologias e Bibliotecas Principais

- ***Python 3.11*:** Linguagem de programação base.
- ***Django 4.x*:** *Framework* web que fornece a estrutura MVC (*Model-View-Controller*), ORM (*Object-Relational Mapper*) e sistema de administração.

- **Django REST Framework (DRF)**: *Toolkit* poderoso para a construção de APIs REST, cuidando da serialização, autenticação, permissões e visualização de *endpoints*.
- **GeoDjango**: Módulo do *Django* que integra funcionalidades SIG, permitindo o uso de campos de modelo geográficos e consultas espaciais no ORM.
- **Celery**: Sistema de fila de tarefas distribuídas para processamento assíncrono.
- **Simple JWT**: Implementa a autenticação baseada em JSON *Web Tokens* (JWT), fornecendo *endpoints* para obtenção, atualização e verificação de *tokens*.
- **drf-spectacular**: Gera automaticamente a documentação da API no formato *OpenAPI* 3, que pode ser visualizada através do *Swagger* ou *Redoc*.
- **Psycopg2**: Adaptador de banco de dados para *PostgreSQL* em *Python*.

A.3.2 Estrutura de Módulos (*Apps*)

O código do *back-end* é organizado em aplicações *Django*, localizadas no diretório `backend/apps/`. Cada *app* encapsula uma funcionalidade de negócio específica:

- **authentication**: Gerencia usuários (**User**), perfis (**UserProfile**), autenticação (*login/logout*) e registro. Utiliza um modelo de usuário customizado.
- **routes**: Modela e gerencia as rotas de coleta (**Route**), seus agendamentos (**RouteSchedule**) e histórico de execuções (**RouteExecution**).
- **vehicles**: Responsável pelo cadastro e gerenciamento de veículos (**Vehicle**), motoristas (**Driver**) e registros de manutenção (**VehicleMaintenance**).
- **collection_points**: Gerencia os pontos de coleta (**CollectionPoint**), os tipos de resíduos que aceitam e o histórico de coletas em cada ponto.
- **collections**: Orquestra o processo de uma coleta, unindo uma rota, veículo e motorista para uma data específica, e registrando os itens de coleta em cada ponto.
- **public_api**: Expõe um conjunto limitado de *endpoints* para consulta pública, sem necessidade de autenticação.

A.3.3 Modelo de Dados (*Models*)

Os principais modelos de dados e seus relacionamentos são:

- **User:** Herda de **AbstractUser** do *Django*, usando o *e-mail* como campo de *login*. Possui papéis (*admin*, *operator*, *viewer*) que podem ser usados para controle de acesso.
- **Route:** Contém um campo **geometry** do tipo **LineStringField** (*PostGIS*) para armazenar o traçado da rota. Relaciona-se com **User** (quem a criou) e **RouteSchedule**.
- **Vehicle** e **Driver:** Modelos para cadastrar a frota e os motoristas, com informações detalhadas como placa, CNH, *status* e datas de manutenção.
- **CollectionPoint:** Possui um campo **location** do tipo **PointField** (*PostGIS*) para as coordenadas geográficas.
- **Collection:** É o modelo central da operação. Ele amarra uma **Route**, um **Vehicle** e um **Driver** a uma data/hora agendada, registrando o progresso e os totais da coleta.
- **CollectionItem:** Detalha a coleta em um ponto específico dentro de uma **Collection**, registrando se o ponto foi coletado, o peso e observações.

A.3.4 API Endpoints

A API é a principal interface de comunicação com o *front-end*. Os *endpoints* são definidos nos arquivos `urls.py` de cada *app* e registrados no `config/urls.py` principal. O DRF com `DefaultRouter` é usado para gerar automaticamente os *endpoints* de CRUD para os `ViewSets`.

- `/api/auth/`: *Endpoints* para registro, *login*, *logout* e gerenciamento de perfil.
- `/api/auth/token/`: *Endpoints* padrão do *Simple JWT* para manipulação de *tokens*.
- `/api/routes/`, `/api/vehicles/`, etc.: *Endpoints* de CRUD para as principais entidades do sistema.
- `/api/docs/`: Interface do *Swagger* para exploração e teste da API.

A.3.5 Script de Inicialização (`entrypoint.sh`)

O *container* do *back-end* utiliza um *script* de entrada (`docker/entrypoint.sh`) que executa tarefas essenciais antes de iniciar o servidor *Django*:

1. Aguarda o *container* do banco de dados (**db**) estar pronto e aceitando conexões na porta 5432.
2. Executa as migrações do banco de dados com `python manage.py migrate`.

3. Coleta todos os arquivos estáticos dos *apps* para um diretório único com `python manage.py collectstatic`.
4. Inicia o servidor de desenvolvimento do *Django*, que escuta em todas as interfaces de rede (0.0.0.0) na porta 8000.

A.4 Front-end

A interface do usuário é uma SPA construída com *React*.

A.4.1 Tecnologias e Bibliotecas Principais

- **React 18**: Biblioteca para construção de interfaces de usuário baseadas em componentes.
- **React Router DOM**: Gerencia a navegação e o roteamento do lado do cliente, permitindo *URLs* distintas para cada página sem recarregar o navegador.
- **Axios**: Cliente HTTP para realizar requisições à API do *back-end*. É configurado com *interceptors* para gerenciar automaticamente a injeção e a renovação de *tokens* JWT.
- **Leaflet & React-Leaflet**: Bibliotecas para a criação de mapas interativos, essenciais para visualizar rotas e pontos de coleta.
- **Chart.js & React-Chartjs-2**: Utilizadas para renderizar gráficos e *dashboards* com estatísticas da operação.
- **Bootstrap & React-Bootstrap**: *Framework* de CSS e conjunto de componentes *React* que agilizam o desenvolvimento de uma interface responsiva e visualmente consistente.
- **React Hook Form**: Para gerenciamento de formulários complexos com validação.
- **JS-Cookie**: Biblioteca para manipular *cookies* de forma simples, usada para armazenar os *tokens* de autenticação.

A.4.2 Estrutura de Componentes e Páginas

O código-fonte do *front-end* reside em `frontend/src` e é organizado da seguinte forma:

- `pages/`: Cada arquivo aqui representa uma tela principal da aplicação, como `Dashboard.js`, `Routes.js`, `Login.js`, etc. Essas páginas compõem as seções principais do sistema e geralmente orquestram vários componentes menores.

- **components/**: Contém componentes reutilizáveis usados em várias páginas. Exemplos incluem `Navigation.js` (a barra de navegação principal), `MapComponent.js` (um *wrapper* para o mapa *Leaflet*), e diversos modais para operações de CRUD (`RouteModal.js`, `VehicleModal.js`).
- **services/**: Contém a lógica de comunicação com serviços externos, principalmente a API do *back-end*.
 - `api.js`: Centraliza toda a configuração do *Axios* e exporta funções para interagir com cada *endpoint* da API (ex: `routesAPI.getRoutes`, `authAPI.login`). O uso de *interceptors* para *refresh* de *token* é uma peça chave aqui, automatizando a renovação de sessões expiradas.
 - `authContext.js`: Implementa um Contexto *React* para gerenciamento do estado de autenticação. Ele provê informações sobre o usuário *logado* e funções como `login`, `logout` e `register` para toda a aplicação.

A.4.3 Roteamento e Autenticação

O arquivo `App.js` define a estrutura de roteamento da aplicação usando `React Router`.

- **Rotas Protegidas (`ProtectedRoute`) e Públicas (`PublicRoute`)**: São componentes de ordem superior (HOCs) que envolvem as rotas para controlar o acesso. `ProtectedRoute` verifica se o usuário está autenticado (usando o `AuthContext`); caso contrário, redireciona para a página de *login*. `PublicRoute` faz o oposto, redirecionando para o *dashboard* se o usuário já estiver logado.
- **Gerenciamento de *Tokens***: O fluxo de autenticação é baseado em JWT. Após o *login*, um *token* de acesso e um *token* de atualização (*refresh token*) são armazenados em *cookies*. O *token* de acesso é enviado em cada requisição no cabeçalho `Authorization`. Se o *token* de acesso expirar (resultando em um erro 401 da API), o interceptor do *Axios* em `api.js` tenta usar o *refresh token* para obter um novo *token* de acesso silenciosamente, sem interromper a experiência do usuário.

A.4.4 *Build* e Implantação

O `Dockerfile.frontend` define o processo de *build* do *front-end*:

1. Utiliza uma imagem base do *Node.js*.
2. Copia o `package.json` e instala as dependências com `npm install`.
3. Copia o restante do código-fonte.

4. Executa o *script* `npm run build`, que traduz o código *React* e o empacota em arquivos estáticos (HTML, CSS, JS) otimizados no diretório `/build`.
5. Instala o `serve`, um servidor estático simples.
6. O comando final serve o conteúdo do diretório `build` na porta 3000.

A.5 Sugestões para Trabalhos Futuros

A arquitetura e a base de código atuais do sistema fornecem um alicerce para diversas expansões e melhorias. Abaixo estão algumas sugestões para trabalhos futuros:

- **Notificações em Tempo Real:** Utilizar *WebSockets* para fornecer *feedback* em tempo real. Por exemplo, notificar os operadores no *dashboard* quando uma coleta é iniciada ou finalizada, ou alertar sobre atrasos. A API pública poderia notificar os cidadãos sobre a proximidade do caminhão de coleta.
- **Aplicativo Móvel para Motoristas:** Desenvolver um aplicativo móvel para os motoristas. O *app* poderia guiá-los pela rota, permitir o registro de coletas (com peso, fotos e observações) de forma *offline* e sincronizar os dados quando houver conexão, além de facilitar a comunicação de incidentes.
- **Análise de Dados e Relatórios Avançados:** Expandir o módulo de relatórios para incluir análises mais profundas e visualizações interativas. Utilizar os dados históricos para aplicar técnicas de *Machine Learning*, prevendo a geração de resíduos por região, otimizando a frequência de coleta e prevendo a necessidade de manutenção dos veículos.
- **Portal do Cidadão:** Aprimorar a `public_api` e criar uma interface dedicada aos cidadãos, onde poderiam consultar o cronograma de coleta para seu endereço, relatar problemas (contêiner danificado, lixo acumulado) e receber dicas sobre descarte correto e reciclagem.

A.6 Links Úteis

- **Aplicação *Front-end*:** Interface principal do sistema: [<http://localhost:3000>](http://localhost:3000)
- **Painel Administrativo do *Django*:** Para gerenciamento dos dados: [<http://localhost:8000/admin>](http://localhost:8000/admin)
- **Documentação da API (*Swagger*):** Interface interativa para explorar e testar os *endpoints* da API: [<http://localhost:8000/api/docs/>](http://localhost:8000/api/docs/)